MODULE 24 · DOMAIN 3 - DEPLOYMENT

Lambda Versions, Aliases and Safe Releases

$LATEST, published versions and the alias that stands between your clients and your deployments.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 24 of 34

admin@sthithapragna.com

A version is **immutable**. An alias is a **movable pointer**. Clients should only ever hold the pointer.

$LATEST and published versions

- **$LATEST** is the mutable working copy. Every code or configuration update changes it.
- **PublishVersion** snapshots $LATEST into a numbered, immutable version.
- Version numbers increment from 1 and are never reused, even after deletion.
- **A published version can never be modified.** To change it, publish a new one.
- Invoking the unqualified function name invokes **$LATEST**, which is almost never what production wants.

> ON THE EXAM
>
> That last line is the root of most problems in this module. An unqualified ARN means every deployment reaches production the instant the code updates.

What a version actually captures

- The **code**, the **runtime**, the **handler** and the **memory and timeout settings**.
- The **environment variables** as they were at publish time.
- The **layers** attached, at their specific layer versions.
- So changing an environment variable requires **publishing a new version** to take effect.
- Not captured: the execution role's **policy**, which is looked up at invocation time.

> ON THE EXAM
>
> The environment variable point catches people constantly. Updating a variable changes $LATEST only; the version your alias points at still holds the old value.

Qualified and unqualified ARNs

- **Unqualified:** ...function:my-function. This resolves to **$LATEST**.
- **Qualified by version:** ...function:my-function**:3**. Pinned to version 3 forever.
- **Qualified by alias:** ...function:my-function**:live**. Follows wherever live points.
- The alias form is what almost everything should use.
- A version-qualified ARN never moves, so a client holding one never receives updates.

ON THE EXAM

Three ARN shapes, three completely different behaviours. Reading the suffix on an ARN in an exam question tells you what will happen when the function is deployed.

Aliases

- A named pointer to one published version. Typical names: **live**, **prod**, **staging**.
- Deploying is **UpdateAlias** to point at a new version. Rolling back is pointing it back.
- An alias has its own ARN, its own permissions and its own CloudWatch metrics.
- **An alias cannot point at $LATEST**, and cannot point at another alias.
- Several aliases can point at the same version, which is how staging and prod share code.

> ON THE EXAM
>
> Rollback being a pointer move is the whole value. The previous version still exists, unchanged, so reverting takes one API call and no redeployment.

Point at a version or an alias?

A VERSION ARN

Pinned forever

- Always runs exactly that code
- Never changes underneath a client
- Updating means editing the caller
- Useful for a deliberate long-term pin
- Wrong for anything you deploy to

AN ALIAS ARN

Stable, and movable

- Stable identifier for the client
- You move it to deploy
- Rollback is one API call
- Carries its own permissions
- The right default for everything

THE TIEBREAKER

Both are qualified ARNs, and only one lets you deploy. A version ARN is stable in the wrong way: nothing changes, including the things you want to change.

Weighted aliases

- A **routing configuration** on an alias sends a percentage of invocations to a second version.
- Specify 10% to the additional version, and the primary automatically receives 90%.
- This is the mechanism CodeDeploy drives during a canary or linear deployment.
- Distribution is **probabilistic**, so at low traffic the actual split can vary noticeably.
- It works for synchronous and asynchronous invocation, and for event source mappings.

ON THE EXAM

This is what module 23's Canary10Percent5Minutes actually does: set a weight of 10%, wait, then move the alias fully. CodeDeploy just automates the two calls.

Weighted alias constraints

- **Maximum two versions.** You cannot split traffic three ways.
- **Both must be published versions.** Neither can be $LATEST.
- Both must have the **same execution role**.
- Both must have the **same dead-letter queue configuration**, or none.
- **A weighted alias cannot have provisioned concurrency.** The two are mutually exclusive.

> READ THIS TWICE
>
> The last one is the least known and the most disruptive: enabling a canary on a function with provisioned concurrency fails outright.

Async invocation and weights

- Weighted routing applies to **asynchronous** invocation and event source mappings too.
- But a **retry** of an async invocation may land on the other version.
- So an event can be processed by version 7 first and version 8 on retry.
- Two versions therefore run against the same data during a canary.
- This is why **backward-compatible changes** matter during a gradual rollout.

> ON THE EXAM
>
> A canary means both versions are live simultaneously. If version 8 writes a field version 7 cannot read, the canary itself causes the failure.

Knowing which version ran

- The **START log line** in CloudWatch includes the version that handled each invocation.
- The **ExecutedVersion** metric dimension splits CloudWatch metrics by version.
- Synchronous responses carry an **x-amz-executed-version** header.
- So error rate can be compared between the canary and the primary directly.
- That comparison is what an alarm-based rollback needs in order to be meaningful.

> ON THE EXAM
>
> Without ExecutedVersion, a canary alarm measures the blend of both versions. Splitting by version is what lets you detect that only the new one is failing.

What should point at an alias

- **API Gateway integrations**, using the alias-qualified ARN or a stage variable.
- **Event source mappings** for SQS, Kinesis and DynamoDB Streams.
- **Function URLs**, which take a qualifier.
- **EventBridge targets** and S3 event notifications.
- Anything pointing at the unqualified function is effectively deployed to instantly.

> ON THE EXAM
>
> Module 11's stage variables pair with this: the dev stage points at the dev alias, prod at the prod alias, from one API definition and one function.

Provisioned concurrency and versions

- **Provisioned concurrency cannot be set on $LATEST.** It requires a version or an alias.
- That is itself an argument for versioning: without it you cannot use the feature at all.
- Configured on an alias, it moves with the alias when you update it.
- New environments must warm up, so a deployment briefly loses the benefit.
- Module 4 covered the cold start side. This is the deployment side of the same feature.

ON THE EXAM

The warm-up gap is real: immediately after moving the alias, the new version's provisioned environments are still initialising, so early invocations can still cold start.

Permissions per qualifier

- A function's **resource policy** can be scoped to a specific version or alias.
- So API Gateway can be permitted to invoke **live** but not **staging**.
- Permission granted on the unqualified function does **not** automatically cover an alias.
- That is a common cause of a 502 after switching an integration to an alias ARN.
- SAM and CDK add the correct qualified permission for you.

> ON THE EXAM
>
> Switching an integration from the function to an alias without adding the matching permission is a classic self-inflicted outage. The ARN changed, so the policy no longer matches.

Layers and code storage

- Layers are versioned too, and a function version pins **specific layer versions**.
- So updating a layer does not change existing function versions. Nothing shifts under you.
- Each account gets **300 GB of code storage per Region**, covering functions and layers.
- Exceeding it returns **CodeStorageExceededException** and new deployments fail.
- The fix is deleting old versions and unused layer versions. Nothing expires on its own.

ON THE EXAM

Publishing a version on every deployment is correct practice and accumulates storage indefinitely. Nothing cleans it up automatically, so eventually deployments simply stop working.

The safe release pattern

01

Deploy code

Update the function. This changes $LATEST only.

02

Publish

PublishVersion freezes it as an immutable numbered version.

03

Shift

Weight the alias so a small share of traffic reaches the new version.

04

Promote or revert

Move the alias fully, or drop the weight back to zero.

ON THE EXAM

Every tool in this course automates these four steps. SAM's AutoPublishAlias does 1 and 2; DeploymentPreference does 3 and 4 through CodeDeploy.

How each tool does this

TOOL	WHAT IT HANDLES
Console or CLI	PublishVersion and UpdateAlias, called by hand
AWS SAM	AutoPublishAlias publishes and repoints. DeploymentPreference shifts traffic
AWS CDK	currentVersion and an Alias construct, with CodeDeploy for shifting
CloudFormation	AWS::Lambda::Version and AWS::Lambda::Alias with a RoutingConfig
CodeDeploy	Drives the alias weights on a canary or linear schedule, with rollback

ON THE EXAM

They all end at the same two API calls. Knowing that makes the tools easier to reason about, and makes a failure in any of them easier to place.

Cleaning up

- Every deployment publishes a version, and versions accumulate forever.
- **A version referenced by an alias cannot be deleted.** Move the alias first.
- Keep the last few versions so rollback targets remain available.
- A scheduled Lambda function is the usual cleanup mechanism. There is no lifecycle rule.
- Deleting a version does not free its number for reuse.

> ON THE EXAM
>
> Keep enough versions to roll back to, and delete the rest. 'How far back might we need to revert' is the question that sets the number.

Versions across environments

- Several aliases on one function can point at different versions.
- So **staging** points at version 9 while **prod** still points at version 7.
- Promotion is then moving prod to the version staging already validated.
- The **same code** runs in both, so environment differences must come from elsewhere.
- Use API Gateway stage variables, or separate accounts, rather than per-environment code.

ON THE EXAM

Promoting the exact version that passed staging is the value here. Rebuilding for production reintroduces the risk that testing was meant to remove.

The common mistakes

- Production pointing at the **unqualified function**, so every deployment is immediate and irreversible.
- Changing an **environment variable** and expecting the alias to see it without publishing.
- Switching an integration to an alias without adding the **qualified invoke permission**.
- Trying to weight an alias that has **provisioned concurrency**.
- Never deleting old versions, until **CodeStorageExceededException** stops all deployments.

> ON THE EXAM
>
> Five mistakes, and four of them produce a failure that looks like something else entirely. That is why this small topic carries so much exam weight.

For each one, ask: what exactly is the client pointing at?

An API Gateway REST API integrates with a Lambda function using the function's ARN with no qualifier. A developer updates the function code to fix a bug in a development account, but the same deployment process runs against production.

Production begins returning errors within seconds of the deployment. Reverting means redeploying the previous code, which takes several minutes.

What change would prevent this?

A Add reserved concurrency to the production function to isolate it

B Pin the API Gateway integration to a specific version number

C Publish a version on each deployment and point the integration at an alias, updated separately

D Enable provisioned concurrency so the old code stays warm

◆ PAUSE AND DECIDE

What change would prevent this?

A Reserved concurrency

Caps how many invocations run at once. It has nothing to do with which code version they run.

B Pin to a version number

Fixes the immediacy and creates a new problem: the integration never moves, so every release means editing and redeploying the API.

C Publish a version, point at an alias

Code updates change $LATEST only, so nothing reaches production until the alias moves. Reverting is moving it back, which is one API call.

D Provisioned concurrency

Addresses cold starts. Warm environments running the broken code fail exactly as fast as cold ones.

THE SIGNAL

Option B is the instructive near miss. It solves the stated problem by removing the ability to deploy at all, which is why the alias exists in between.

> A team runs a latency-sensitive function with provisioned concurrency configured on its live alias. They want to add canary deployments, shifting 10% of traffic to each new version for five minutes.
>
> Configuring the routing weight fails with InvalidParameterValueException, stating that an alias with weights cannot be used with provisioned concurrency.

What is the correct interpretation?

A. The two features are mutually exclusive on the same alias, so the design must choose one or restructure

B. Provisioned concurrency must be increased before weights can be applied

C. The alias must point at $LATEST for weighted routing to work

D. Weighted routing requires the function to have reserved concurrency configured first

◆ PAUSE AND DECIDE

What is the correct interpretation?

A Mutually exclusive

The error is stating a hard constraint, not a misconfiguration. A weighted alias cannot carry provisioned concurrency, so one of the two requirements must give.

B Increase provisioned concurrency

Reads a constraint as a capacity problem. No amount of provisioned concurrency makes the combination valid.

C Point at $LATEST

Aliases cannot point at $LATEST at all, and weighted routing specifically requires two published versions.

D Reserved concurrency first

Reserved and provisioned concurrency are different features. Neither enables weighted routing.

THE SIGNAL

Some exam answers are simply 'this combination is not supported'. Recognising a hard constraint rather than hunting for a workaround is the skill being tested.

> A team's CI/CD pipeline has published a version on every deployment for two years across roughly 60 functions. Deployments now fail with CodeStorageExceededException, and no new functions can be created in the Region.
>
> Rollback capability to recent releases must be retained.

What should the team do?

A. Request a quota increase for Lambda concurrent executions

B. Stop publishing versions and deploy to $LATEST instead

C. Add a lifecycle rule to expire old function versions automatically

D. Delete old versions that no alias points at, keeping the most recent few per function

◆ PAUSE AND DECIDE

What should the team do?

A Concurrency quota increase

Concurrency and code storage are unrelated quotas. This error is about stored bytes, not simultaneous executions.

B Deploy to $LATEST

Frees storage by abandoning versioning, which discards rollback, canary deployments and provisioned concurrency in one move.

C A lifecycle rule

No such feature exists for Lambda versions. Nothing expires automatically, which is why the storage accumulated.

D Delete unreferenced old versions

The 300 GB per-Region quota covers functions and layers. Deleting versions no alias points at frees space while keeping recent rollback targets.

THE SIGNAL

Option C is worth remembering as a false feature. The absence of automatic expiry is exactly why this problem happens at all.

A function reads an API endpoint from an environment variable. A developer updates that variable to point at a new endpoint and confirms the change is saved in the console. Traffic continues going to the old endpoint.

The function is invoked through an alias named live, which points at version 7. The deployment pipeline was not run.

Why has the change not taken effect?

A Environment variable changes require the function to be restarted

B The variable changed on $LATEST only. Version 7 retains the value captured when it was published

C The alias is caching the old configuration and needs to be recreated

D Environment variables cannot be changed after a version is published

◆ PAUSE AND DECIDE

Why has the change not taken effect?

A Needs a restart

Lambda has no restart concept, and new execution environments start constantly. They would pick up a change if the version had one.

B Changed on $LATEST only

A version freezes its environment variables at publish time. Editing them changes $LATEST, and version 7 is unaffected.

C Alias caching

An alias is a pointer, not a cache. It resolves to version 7, which genuinely holds the old value.

D Cannot be changed after publishing

Half right in a misleading way. You cannot change them on a published version, and you can change them on $LATEST, which is what happened.

THE SIGNAL

Option D states a true fact and draws the wrong conclusion. The console let the edit succeed because it applied to $LATEST, not to version 7.

What to carry forward

- **Versions are immutable, aliases move.** Clients hold the alias, never the version.
- **An unqualified ARN means $LATEST**, so every deployment is instant and irreversible.
- **A version freezes environment variables and layer versions**, not just code.
- Weighted aliases: two published versions maximum, and never with provisioned concurrency.
- 300 GB of code storage per Region, and nothing expires on its own.

> ON THE EXAM
>
> That closes the deployment services. Next: CloudWatch logs, metrics and alarms, opening the observability run into Domain 4.